实验 4_预训练语言模型实现与应用

提交人：GaoT

日期：2026 年 3 月

一、 代码问题回答

1. 这里的 fact_in_annotated_train 代表什么？(gen_data.py L82)

   fact_in_annotated_train 是一个集合，存储了训练集中所有已标注的（实体对，关系）三元组。

   - 每个元素是 (头实体名称, 尾实体名称, 关系) 的三元组
   - 对于训练集中的每个正例关系标签，会将该实体对和对应关系的所有可能组合都加入集合
   - 这个集合用于后续在验证集和测试集中标记某个关系实例是否在训练集中出现过（通过 in_annotated_train 字段），这在某些情况下可能用于分析或过滤

2. 这里的 na_triple 代表什么？(gen_data.py L99)

   na_triple 存储了所有负例的实体对索引。

   - 遍历文档中所有实体对（排除自己与自己）
   - 如果该实体对 (j, k) 不在训练集的正例集合 train_triple 中
   - 就将其作为负例（NA 关系）加入 na_triple
   - 这些负例用于模型训练时的负采样，表示这些实体对之间没有关系

3. 这里的 bert_starts_ends 代表什么？(gen_data.py L126)

   bert_starts_ends 是一个三维数组，存储了每个原始单词在 BERT token 序列中的起止位置映射。维度为 (句子数, max_seq_length, 2)，其中：

   - 第三个维度的 0 位置存储该单词的起始 token 索引
   - 第三个维度的 1 位置存储该单词的结束 token 索引
   - 这个映射关系用于后续实体识别或关系抽取时，将原始文本中的实体位置定位到 BERT token 序列中的位置

4. token_start_idxs 的公式中为什么要加 1？(gen_data.py L141)

   加 1 是因为 BERT 的输入序列开头添加了[CLS]标记。

   - tokens = [tokenizer.cls_token] + flatten_subwords[: max_seq_length - 2] + [tokenizer.sep_token] 在开头添加了[CLS]标记
   - np.cumsum([0] + subword_lengths[:-1]) 计算每个单词的 subword 在整体序列中的累积位置（从 0 开始），没有考虑开头的[CLS]标记
   - 所以需要加上 1 将位置索引向后偏移一位，得到该单词在 BERT 输入序列 tokens 中的实际起始位置

5. 为什么要把 token_start_idxs >= max_seq_length-1 的都置为 max_seq_length – 1？(gen_data.py L143)

   为了防止索引超出序列长度。

   - BERT 输入的最大长度是 max_seq_length，序列末尾有[SEP]标记
   - 最后一个有效位置是 max_seq_length - 1（[SEP]的位置）

6. data_train_bert_mask 代表什么？（Config.py L44）

   标记 BERT 输入序列中哪些位置是有效 token（值为 1），哪些是 padding（值为 0）。用于 attention 机制，避免模型关注 padding 位置。

7. 这里的 neg_multiple 代表什么？（Config.py L89）

   neg_multiple 是负例采样倍数，控制训练时负例的数量。值为 3 表示如果文档有 1 个正例，则采样 3 个负例。作用是平衡正负样本，防止模型偏向预测 NA。

8. 为什么要设置一个 h_t_limit？（Config.py L97）

   为了限制每个文档处理的实体对最大数量。当文档实体过大时，可以防止显存溢出，同时过滤掉不重要的实体对。

9. 这里的 self.dis2idx 代表什么？（Config.py L102）

   将实体间的距离映射到离散的位置编码，让模型感知实体间的相对距离。

10. h_mapping 和 t_mapping 有什么区别？（Config.py L160）

    h_mapping 用于头实体的位置编码，而 t_mapping 用于尾实体的位置编码。

11. max_h_t_cnt 有什么作用？（Config.py L180）

    记录批次中最大的实体对数量。用于动态计算批次中实际使用的实体对数量，避免分配过大的张量，节省显存。

12. 本代码是如何处理 token 数量大于 512 的文档的？(Config.py L209)

    采用硬截断策略，直接丢弃超出长度的实体。具体方法是：只保留结束位置 < 511 的实体提及（511 是[SEP]的位置），如果实体的所有提及都被截断，则跳过该实体对。

13. 为什么要先除以 len(hlist)， 再除以 (h[1] - h[0])？(Config.py L221)

    - 除以 len(hlist)：均匀分配权重给每个提及
    - 除以(h[1] - h[0])：将提及的总权重均匀分配给其覆盖的每个 token

    最终每个 token 的权重 = 1 / (提及数 × 提及长度)

14. relation_mask 的作用是什么？(Config.py L227)

    标记该实体对是有效实体对，用于 loss 计算时忽略 padding 位置。

15. max_h_t_cnt 代表什么？(Config.py L377)

    代表训练集批次中最大的实体对数量，包括正例数和采样的负例数。

16. 这里可以使用 cross-entropy loss 吗？(Config.py L416)

    不能。一个实体对可能有多个关系标签（多标签分类），BCEWithLogitsLoss 支持多标签，而 CrossEntropy 只能处理单标签。

17. 这里的 NA acc / not NA acc / tot acc 分表代表什么？是如何计算的？(Config.py L522)

- NA acc：真实标签为 NA（关系 0）的样本的预测准确率。
- not NA acc：真实标签为非 NA（关系 1-96）的样本的预测准确率。
- tot acc：所有样本的预测准确率。

预测模型输出$predict_re$，取每个位置概率最大的关系 ID 获得$output$（[batch_size, h_t_limit]），遍历 output 的元素并与真实关系 relation_label 比较，对正确预测结果进行统计：

```
for i in range(output.shape[0]):        # batch内每个文档
    for j in range(output.shape[1]):    # 每个实体对
        label = relation_label[i][j]    # 真实标签
        if label < 0:                   # 跳过padding
            break

        if label == 0:  # 真实标签为NA的情况
            self.acc_NA.add(output[i][j] == label)
        else:  # 真实标签为非NA的情况
            self.acc_not_NA.add(output[i][j] == label)

        self.acc_total.add(output[i][j] == label)
```

18. 这里是否可以改成 softmax 函数？(Config.py L608)

不能改成 softmax 函数，因为 DocRED 是一个多标签分类问题，一个实体对可能同时存在多个关系。Sigmoid 允许每个关系独立判断，而 Softmax 强制只能选一个关系，不符合数据集的特性。

19. 这里的 Theta / F1 / AUC 分别代表什么？是如何计算的？(Config.py L692)

- Theta：最佳决策阈值，使 F1 分数达到最大值的预测概率阈值。

  计算方法：(1) 找到最大 F1 对应的位置 $f1_pos = f1_arr.argmax()$

  (2) 该位置对应的预测分数就是最佳阈值 $theta = test_result[f1_pos][1]$
- F1：最佳/最大 F1 分数，其中 F1 分数是 Precision 和 Recall 的调和平均数。

  计算方法：(1) 计算所有位置 $f1_arr = 2 \times Precision \times Recall / (Precision + Recall)$

  (2) 取最大值 $f1 = f1_arr.max()$
- AUC：PR 曲线下的面积，不受类别不平衡影响，值越大表示模型性能越好（0.5~1.0）。

  计算方法：使用 sklearn 计算 $auc = sklearn.metrics.auc(x = pr_x, y = pr_y)$

20. 这里的 input_theta / test_result F1 / AUC 分别代表什么？是如何计算的？(Config.py L695)

- input_theta：用户指定的预测阈值（通过命令行参数 --input_threshold 传入）；如果用户不指定（验证集/开发集测试时），将输出最佳阈值（即上一问的 Theta）。
- test_result F1：在指定阈值（input_theta）下的 F1 分数；同样用户不指定的情况下为上一问的最大 F1 分数。

  计算方法：(1) 计算所有位置 $f1_arr = 2 \times Precision \times Recall / (Precision + Recall)$

  (2) 找到 input_theta 对应（大于 input_theta 的最小预测阈值）的位置 w

  (3) 取该位置对应的 F1 $test_result_f1 = f1_arr[w]$
- AUC：PR 曲线下的面积（与上一问的 AUC 是同一值）。

这几个评价指标是在测试阶段计算的，而上一问是在训练阶段为了观察模型在测试集上的表现而计算的。

21. 这里的 input_theta / test_result F1 / AUC 分别代表什么？是如何计算的？(Config.py L726)

这几个指标与上一问的计算逻辑一致，只不过在计算$Precision$时排除了训练集中已经出现过的关系，只评估模型在"新关系"上的泛化能力。

- input_theta：与上一问完全一致。
- test_result F1：计算逻辑完全一致，但是$Precision$的计算需要排除训练集中已经出现过的关系。$Precision = (correct - seen_in_train) / (i + 1 - seen_in_train)$
- AUC：与上一问逻辑一致，计算时使用排除训练集已出现过的关系的$Precision$。

二、 模型训练和评价

batch_size 默认值为 32，由于 GPU 限制，本实验采用更小的 batch_size 并配合梯度累计：

```
batch_size =4
gradient_accumulation_steps = 8
```

预训练模型除了 BERT，还尝试使用 RoBERTa，并对最终模型性能进行比较。

对于两个预训练模型都分别采用了 10%、50%和 100%的随机采样进行训练，Epoch 统一设置为 100。

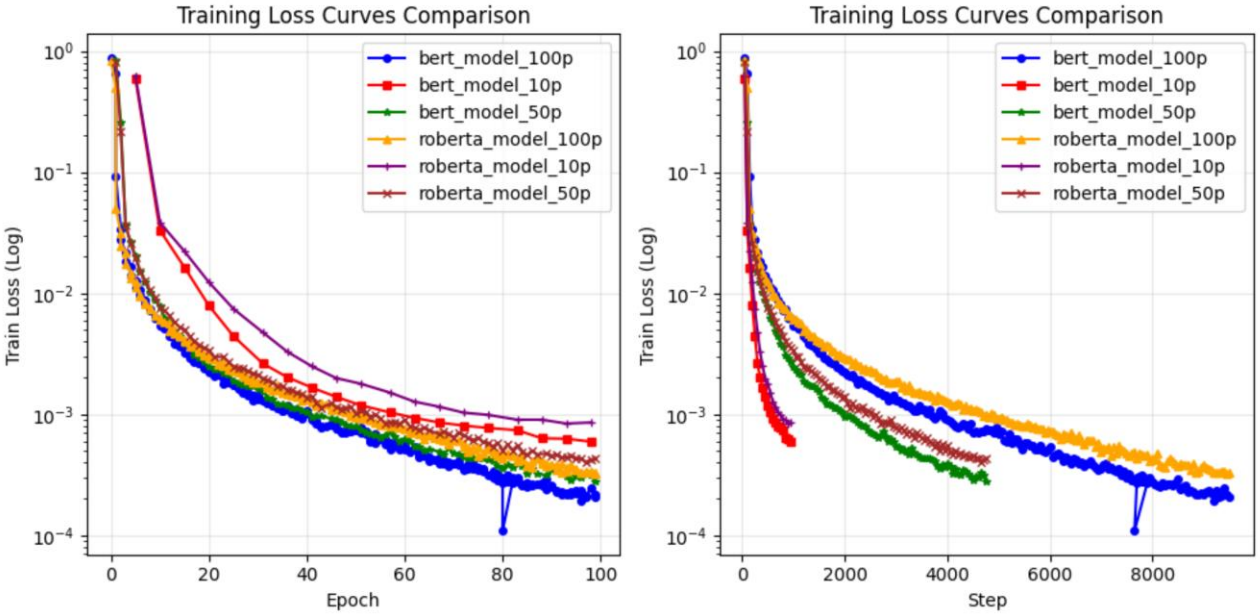
1. 开发集评价指标

预训练模型	训练集百分比	阈值 Theta	Best F1	AUC	@Epoch	Ign_F1	Ign_AUC
BERT	10%	0.4890	0.4249	0.3665	99	0.4004	0.3391
BERT	50%	0.7367	0.5332	0.5090	99	0.5101	0.4763
BERT	100%	0.8666	0.5662	0.5534	99	0.5439	0.5209
RoBERTa	10%	0.7951	0.4533	0.3980	89	0.4342	0.3700
RoBERTa	50%	0.9284	0.5444	0.5248	99	0.5247	0.4943
RoBERTa	100%	0.8949	0.5731	0.5611	84	0.5510	0.5278

- 同样的预训练模型和超参数，训练集数据越多模型性能越好，但 50%训练集的性能与 100%已经很接近，在计算资源不足的时候可以选择 50%训练集。
- RoBERTa 在同样的超参数设置下比 BERT 的性能更好。

2. 训练损失曲线

```
bert_model_100p.pt: 100 个 Epoch, 9501 个 Step, 最终 loss = 0.000206
bert_model_10p.pt: 99 个 Epoch, 951 个 Step, 最终 loss = 0.000594
bert_model_50p.pt: 100 个 Epoch, 4751 个 Step, 最终 loss = 0.000277
roberta_model_100p.pt: 100 个 Epoch, 9501 个 Step, 最终 loss = 0.000328
roberta_model_10p.pt: 99 个 Epoch, 951 个 Step, 最终 loss = 0.000855
roberta_model_50p.pt: 100 个 Epoch, 4751 个 Step, 最终 loss = 0.000432
```



- 相同的 epoch 训练数据量大，loss 收敛越快。（左侧 loss 曲线）
- 采用相同的 batch_size，训练数据量小的模型 step 数小，相同的 step 的情况下 loss 收敛快。（右侧 loss 曲线）
- 训练数据相同的情况下，RoBERTa 的模型效果（评价指标）比 BERT 好，但是 loss 却比 BERT 高。可以认为 RoBERTa 的 loss 曲线更稳定，如果增加 epoch 最终模型性能也应该超过 BERT。

3. 测试集推理

- 使用 BERT（100%训练集）

```
python test.py --model_type bert
               --model_name_or_path bert-base-cased
               --prepro_data_dir prepro_data
               --save_name bert_model_100p.pt
               --test_prefix test
               --input_threshold 0.8666
```

生成推理文件： ***bert_model_100p.pt_test_index.json***

- 使用 RoBERTa（100%训练集）

```
python test.py --model_type roberta
               --model_name_or_path roberta-base
               --prepro_data_dir prepro_data_roberta
               --save_name roberta_model_100p.pt
               --test_prefix test
               --input_threshold 0.8949
```

生成推理文件： ***roberta_model_100p.pt_test_index.json***